



PYTHON PROGRAMMING VIRTUAL INTERNSHIP

Project Presentation

Student: Veronica Bhat

Program: B.Tech CSE • Batch 2025–2029

Institute: School of Computer Science and Engineering, IILM University

**AI +
GENERATIVE AI**

Task Portfolio

UpStartInterns

Online / Remote

Internship Overview

Internship

Python Programming Virtual Internship
Practical, task-based learning with hands-on project work.

Organization

UpStartInterns
A structured internship platform focused on learning by doing.

Academic Context

B.Tech CSE
School of Computer Science and Engineering
IILM University, Greater Noida

Internship Focus

The completed work progressed from prompt engineering and structured model outputs to retrieval-based systems and a grounded assistant. The projects emphasize reliable AI behavior, validation, grounding, evaluation, and practical Python implementation.

Project Description

- Build small, practical AI/Generative AI systems using Python.
- Improve model reliability through better prompts and structured outputs.
- Use retrieval and embeddings to ground answers in provided documents.
- Add guardrails so the system can refuse unrelated or unavailable requests.
- Evaluate the system with fixed test questions and measure approximate usage.

01

Prompt Engineering

Improve prompts

02

Structured Output

Reliable JSON

03

RAG

Retrieve relevant text

04

Grounded Assistant

Ground + evaluate

Task 01 — Prompt Engineering

Goal

Take one task and improve the prompt until the output is reliably good.

The task required multiple prompt versions, a fixed test set, and a note explaining what changed.

Work Completed

- Created and compared prompt versions
- Used fixed test inputs
- Improved the prompt based on output quality
- Recorded the final decision and reasoning

Key Learning

Prompt quality directly affects model output. Iterative testing makes prompts more consistent and helps identify which instructions actually improve results.

Task 02 — Structured Output

Goal

Make a model return JSON that code can safely rely on.

Required elements included a schema, validation, and a retry/fallback path.

Approach

- Define the expected JSON structure
- Validate the returned data
- Reject malformed output
- Handle failure with a retry or fallback

Key Learning

Generative models can produce useful text but their structure cannot be assumed. Validation is essential when model output is consumed by software.

Task 03 — Retrieval-Augmented Generation

Problem

A model may not know specific information contained in a private or provided document. RAG addresses this by retrieving relevant information before answering.

Project

Built a simple college library information assistant using the document `college\_rules.txt` as the grounding source.

1. Load

Read the document from the `docs` folder and split it into usable passages.

2. Embed

Used SentenceTransformer `all-MiniLM-L6-v2` to convert text into embeddings.

3. Retrieve

Compared the question embedding with document passages and returned the most relevant passage.

Task 04 — College Grounded Assistant

- Built a grounded assistant that answers college-related questions from provided documents.
- Displays the answer together with the source document and retrieved passage.
- Uses basic guardrails to refuse unrelated questions.
- Responds safely when requested information is not present in the grounding document.
- Includes an evaluation script and approximate token-usage reporting.

Grounding Source

docs/college_rules.txt

Model

SentenceTransformer
all-MiniLM-L6-v2

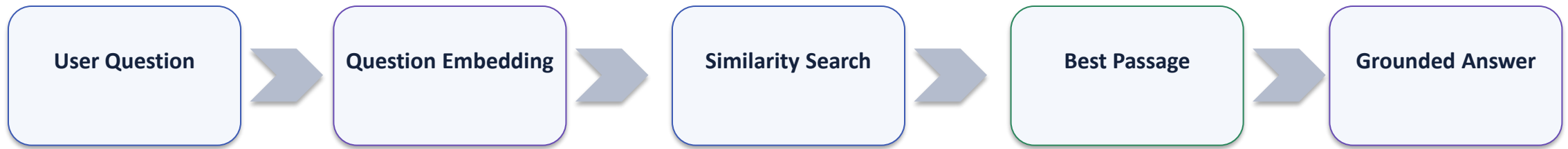
Output

Answer + Source + Retrieved Passage

System Architecture

8

Document Grounding



College rules document → sentence-level passages → embeddings → similarity scoring → relevant passage → transparent response
Guardrails and evaluation operate around the retrieval pipeline.

Evaluation & Results

How many books can a student borrow?

Three books

PASS

What is the library fine?

Rs. 2 per day

PASS

What are the library timings?

8:00 AM to 8:00 PM

PASS

What is the cafeteria menu?

Not found

PASS

Evaluation set: 4 fixed questions • 4/4 passed

100%

Technologies & Tools

Python

Core implementation language

Sentence Transformers

Text embeddings and semantic retrieval

NumPy

Vector similarity calculations

RAG

Grounded retrieval-based answering

Prompt Engineering

Iterative prompt improvement

JSON Validation

Reliable structured outputs

GitHub

Versioning and public submission

Challenges & Learnings

Biggest Challenges

- Making prompts produce consistent outputs
- Ensuring JSON responses are usable by code
- Retrieving the correct passage from a small document
- Handling questions whose answers are not present
- Keeping the system grounded instead of guessing

What I Learned

- Prompt iteration improves reliability
- Structured output needs validation
- Embeddings represent text as vectors of meaning
- RAG can ground answers in provided information
- Guardrails help control scope
- Evaluation turns assumptions into measurable results

Conclusion

From prompts to grounded AI

The internship provided a practical progression from improving model instructions to building a document-grounded assistant. The work demonstrated how prompt engineering, structured outputs, retrieval, embeddings, guardrails, and evaluation can work together to make AI systems more reliable.

Thank you